Hybrid Evolutionary Optimization for Turbofan Remaining Useful Life Prediction and Fractional-Order Control Tuning

Leonardo H. Talero-Sarmiento *
Universidad Autónoma de Bucaramanga (UNAB), Industrial Engineering Program

Feisar E. Moreno-Corzo †
Universidad Autónoma de Bucaramanga (UNAB), Systems Engineering Program

Francisco Pérez-Galarce ‡
Núcleo de Investigación en Data Science, Facultad de Ingeniería y Negocios,
Universidad de Las Américas, Sede Providencia, Manuel Montt 948, Santiago, Chile

March 25, 2026

Abstract

Predictive maintenance and robust control are twin pillars of modern industrial reliability. This paper presents a hybrid framework that synergizes data-driven prognostics with control-theoretic optimization. Acá sugiero indicar el gap de literatura, por ejemplo, Recently, several large deep learning models have been proposed to address this task. However, such models typically require substantial effort for hyperparameter optimization and training, and often impose significant computational and capacity constraints at inference time. These limitations can be prohibitive in control-oriented architectures or resource-constrained decision-making settings. To advance in a different direction, we propose a lightweight Artificial Neural Network (ANN) for predicting the Remaining Useful Life (RUL) of turbofan engines, utilizing the NASA C-MAPSS dataset. Unlike recent heavy deep learning architectures, our approach employs Particle Swarm Optimization (PSO) to dynamically tune the ANN architecture, balancing accuracy with computational efficiency. Furthermore, the evolutionary optimization framework is extended to the control domain, where a Fractional-Order PID (FOPID) controller is tuned via PSO to ensure stability in a simulated load-frequency system. The methodology allows for accurate degradation monitoring—validated across four complexity levels (FD001–FD004)—and demonstrates how prognostic data can inform robust control actions. The findings suggest that, for prognostic-driven decision-making and control applications, architectural optimization and system integration may be more critical than increasing representational depth.

*Email: `ltalero@unab.edu.co`
†Email: `fmoreno4@unab.edu.co`
‡Corresponding author: `francisco.perez@udla.cl`

Keywords: Remaining Useful Life prediction, NASA C-MAPSS, particle swarm optimization, artificial neural networks, mixed-integer linear programming, model pruning

1 Introduction

The paradigm shift toward Industry 4.0 has significantly increased the relevance of Prognostics and Health Management (PHM) as a core component of intelligent industrial systems. Accurate estimation of the Remaining Useful Life (RUL) of critical assets—such as aircraft engines—enables a transition from reactive and preventive maintenance toward predictive strategies, reducing unplanned downtime, maintenance costs, and the risk of catastrophic failures [1]. In parallel, modern industrial plants operate under progressively harsher and more uncertain conditions, where system degradation and parametric drift challenge conventional control strategies. These trends motivate the need for integrated approaches that jointly address *measurement* (health assessment) and *control* (robust closed-loop behavior).

The NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) dataset has emerged as a widely studied benchmark [1] for RUL prediction research. Early contributions relied on physics-based and statistical models; however, recent advances have been dominated by data-driven methods. Lei et al. [2] extensively documented this transition toward machine learning formalisms designed to trace complex degradation paths. In particular, recurrent neural networks have demonstrated strong capabilities for modeling temporal degradation. Wu et al. [3] established the efficacy of long short-term memory (LSTM) architectures over standard feed-forward networks, an observation structurally consistent with Peringal et al. [4] who noted that LSTM networks outperform classical Multi-Layer Perceptrons (MLPs) by explicitly capturing sequential dependencies. Subsequent studies have further increased architectural complexity: Elsherif et al. [5] proposed a Convolutional Autoencoder combined with an attention-based LSTM (CAELSTM) to learn spatial–temporal features, while Chen [6] introduced a Transformer-based architecture with position-sensitive attention to handle long-term dependencies, parallel to findings by Mo et al. [7].

As network depth increases, achieving optimal performance mathematically necessitates robust hyperparameter optimization (HPO). A concise examination of HPO methodologies for neural networks illustrates a structured progression [8]. Grid Search combined with Cross-Validation offers exhaustive parameter grid evaluation but scales poorly against high dimensionality. Alternatively, Random Search coupled with Cross-Validation reliably samples independent network parameters, frequently isolating effective topologies under tighter computational constraints. To navigate highly parameterized continuous spaces systematically, Bayesian Optimization employs surrogate probabilistic models to intelligently navigate parameter spaces without exhaustive iterations, an approach formalized by Snoek et al. [9]. In the prognostics domain, Xue et al. [10] recently implemented Bayesian approaches to structure multi-modal Transformer–LSTM models, effectively automating architecture selection while operating with substantial computational complexity.

These trajectories expose a critical bipartite gap: the tension between pursuing maximum predictive accuracy through increasing network depth and the requisite computational tractability for deployment. The exponential scaling of deep architectures, such as massive Transformers and CAELSTM models, incurs substantial environmental costs in

terms of energy consumption and carbon footprint, giving rise to the imperative of "Green AI" [11–16]. Heavy networks require specialized hardware, meaning that their deployment implicitly scales up physical, real-world resource consumption—including significant electrical power and water for data-center cooling. Conversely, advancing intelligent sensing requires Artificial Neural Networks (ANNs) that are lightweight enough to be embedded locally on edge devices or small micro-controllers without relying on centralized, large-scale computing [17–22]. Furthermore, while extensive research focuses on refining RUL predictions in isolation, there remains a notable absence of unified evolutionary frameworks that link measurement insights directly to robust control tuning.

To bridge this gap, this paper introduces a holistic architecture that couples a highly optimized, lightweight prognostic model with a robust control mechanism, actively prioritizing computational efficiency and environmental sustainability. Building on this premise, a single Particle Swarm Optimization (PSO) engine is employed to address two tightly related tasks: (i) automated architecture selection for a computationally tractable and environmentally viable MLP-based RUL predictor, and (ii) parameter tuning of a Fractional-Order PID (FOPID) controller. Instead of escalating network depth, we demonstrate that a precisely tuned, shallow ANN operating on explicit temporal embeddings is capable of capturing essential degradation trajectories while remaining small enough for practical edge deployment. Inspired by the work of Gong et al. [23], the FOPID controller introduces additional degrees of freedom through fractional integration and differentiation orders, enhancing robustness under nonlinear and degraded operating conditions. By coupling optimized health estimation with optimized feedback control, the proposed framework advances a holistic interpretation of *Measurement and Control*, in which prognostic information is not an endpoint but a functional input to closed-loop system design.

Se pasa de la introducción a los materiales y métodos en mi opinión, deberíamos crear una breve revisión de literatura donde se muestren: (1) los avances de machine learning y deep learning en la estimación de RUL (2) estimaciones de hiperparámetros en redes neuronales, considerando métodos tradicionales como grid search cross validation, random search cross validation y Bayesian Optimization (3) fractional order PID. Yo recomendaría incorporar referencias de journals Q1 WoS.

recomiendo agregar un párrafo con el outline del paper

Podría ser buena idea agregar un pequeño esquema donde se explique el problema de control que se busca resolver, destacando dónde operaría el modelo y justificando porque es relevante que sea un modelo liviano.

2 Materials and Methods

2.1 Dataset Characterization and Preprocessing

2.1.1 NASA C-MAPSS Turbofan Degradation Dataset

This study uses the NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) benchmark [1], which provides run-to-failure trajectories for turbofan engines under controlled degradations. Each observation contains a unit identifier, a cycle index, three operational settings, and 21 sensor measurements, for a total of 26 columns. The four standard subsets (FD001–FD004) are retained in order to evaluate the proposed workflow under progressively harder combinations of operating-regime variability and

fault diversity.

Table 1: Empirical summary of the C-MAPSS subsets used in this study. Train/test engine counts follow the released files; for FD004 the bundled readme swaps the counts, but the actual files contain 249 training engines and 248 test engines.

Dataset	Op. Conditions	Fault Modes	Train/Test Engines	Train Cycles per Engine	Test Cycles per Engine	Official Test RUL
FD001	1	1	100 / 100	206.31 ± 46.34	130.96 ± 53.59	75.52 ± 41.76
FD002	6	1	260 / 259	206.77 ± 46.78	131.24 ± 63.09	81.19 ± 53.88
FD003	1	2	100 / 100	247.20 ± 86.48	165.96 ± 86.89	75.32 ± 41.60
FD004	6	2	249 / 248	245.98 ± 73.11	166.19 ± 91.61	86.55 ± 54.63

These descriptive statistics confirm the benchmark taxonomy. FD001 and FD003 correspond to single-regime subsets, whereas FD002 and FD004 contain six operating conditions. The column-level audit of the released files makes this distinction explicit: the operational-setting columns have near-zero spread in FD001 and FD003, but wide variation in FD002 and FD004, with standard deviations of approximately 14.75 for `col_3`, 0.31 for `col_4`, and 14.25 for `col_5`. The two-fault subsets (FD003 and FD004) also exhibit longer and more variable degradation histories, with mean training lengths near 246–247 cycles, compared with approximately 206 cycles for FD001 and FD002.

2.1.2 Feature Selection, Regime Handling, and Training-Derived Normalization

The preprocessing pipeline follows the raw C-MAPSS schema documented in the dataset audit. Columns `col_1` and `col_2` correspond to the unit identifier and cycle counter, respectively, and are not used as direct predictors. Columns `col_3`–`col_5` are the three operational settings, while columns `col_6`–`col_26` map to sensor measurements 1–21.

A compact sensor subset is retained in order to keep the ANN lightweight enough for repeated screening inside the optimization loop. Following the literature-derived 10-sensor baseline reviewed in this study, Sensors $\{2, 6, 7, 8, 11, 13, 14, 19, 20, 21\}$ are preserved and Sensors $\{1, 3, 4, 5, 9, 10, 12, 15, 16, 17, 18\}$ are removed. Under the code naming convention, this fixed sensor filter corresponds to dropping `col_6`, `col_8`, `col_9`, `col_10`, `col_14`, `col_15`, `col_17`, `col_20`, `col_21`, `col_22`, and `col_23`. The descriptive statistics partially support this decision: in FD001 and FD003, `col_6`, `col_10`, `col_21`, and `col_23` are constant, and `col_15` is constant or nearly constant. However, several of these channels vary more strongly in FD002 and FD004, so this fixed sensor filter should be interpreted as a conservative, literature-aligned baseline rather than a universal claim that every removed channel is non-informative in all subsets.

The treatment of the operational settings is deliberately dataset-aware. In FD001 and FD003, the settings are dropped because their empirical spread is effectively zero and they add little information beyond noise. In FD002 and FD004, the same variables are retained because they encode the six operating regimes that materially affect engine behavior. This choice aligns the implementation with both the descriptive statistics of the local files and the benchmark definition reported by NASA.

After feature selection, the default pipeline applies a training-derived global standardization rather than a per-trajectory rescaling. Let $\boldsymbol{\mu}_{\text{train}}$ and $\boldsymbol{\sigma}_{\text{train}}$ denote the feature-wise mean and standard deviation estimated from the training portion of the official training split. For unit i at cycle c, with retained feature vector $\mathbf{x}_{i,c} \in \mathbb{R}^d$, the normalized vector is

$$\tilde{\mathbf{x}}_{i,c} = \frac{\mathbf{x}_{i,c} - \boldsymbol{\mu}_{\text{train}}}{\boldsymbol{\sigma}_{\text{train}}}, \tag{1}$$

The same fitted transformation is then applied to the corresponding validation engines and, for the final benchmark run, to the official test split. This choice was adopted for three methodological reasons. First, it preserves a common reference frame across engines, which is desirable when the optimization stage compares architectures on pooled windowed samples. Second, it avoids a mismatch between full training trajectories and truncated official test trajectories: a per-unit Min–Max transformation depends on each engine's own extrema, which are not equally informative in the shortened test runs. Third, compared with a global Min–Max mapping, z-score normalization is less sensitive to extreme values that can compress the dynamic range of the remaining samples. For reproducibility, the code still exposes global Min–Max and per-unit Min–Max alternatives, but the default workflow uses training-derived standardization.

2.1.3 Piecewise-Linear RUL Target Definition

For the run-to-failure training trajectories, the target Remaining Useful Life is computed from the last observed cycle of each engine. Let $C_i^{\max}$ denote the final cycle of unit i. The uncapped RUL at cycle c is $C_i^{\max} - c$. Following the standard piecewise-linear convention in the C-MAPSS literature, the target is saturated at a maximum threshold $R_{\max}$ in order to reduce the influence of the long healthy phase:

$$y_{i,c} = \min\left(C_i^{\max} - c,\ R_{\max}\right). \tag{2}$$

In the implementation, $R_{\max} = 125$ cycles. For the official test split, the benchmark-provided RUL labels are used only after model selection is complete.

Equation (2) can be interpreted with a simple example. Suppose an engine fails at cycle $C_i^{\max} = 220$. Its uncapped target at cycle $c = 20$ would be $220 - 20 = 200$, but the capped target becomes $\min(200, 125) = 125$. At cycle $c = 170$, the uncapped target is 50, which remains unchanged because it is already below the cap. Thus, the piecewise-linear target preserves a long constant plateau during the healthy phase and then transitions into a linear countdown near failure, which reduces the tendency of the model to over-emphasize very early cycles that carry weak degradation information.

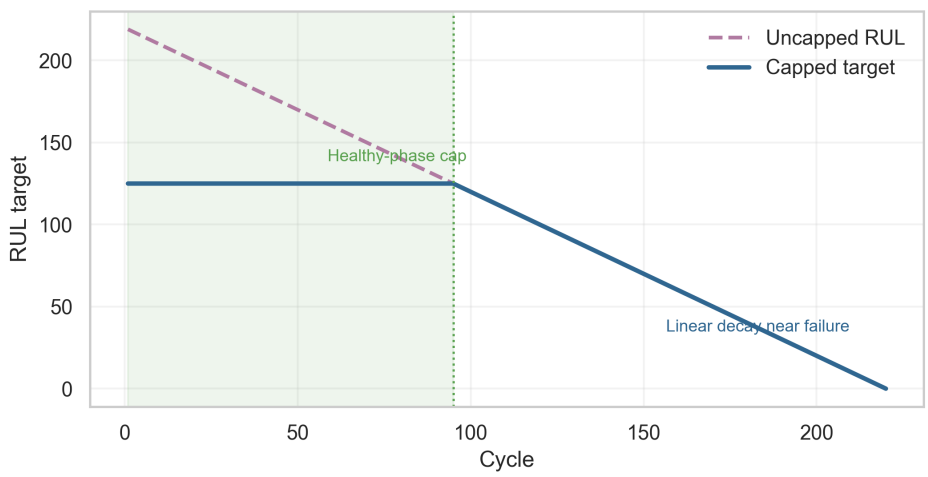

Figure 1: Illustrative capped RUL target for one synthetic run-to-failure trajectory. The plateau corresponds to the capped healthy phase, while the descending branch represents the linear countdown near failure.

2.1.4 Sliding-Window Construction and Flattened Temporal Embeddings

To encode short-term degradation dynamics without recurrent layers, each engine trajectory is transformed into overlapping fixed-length windows. With a window length $L = 30$ and d retained variables after feature filtering, the input associated with unit i and cycle t is

$$\mathbf{s}_{i,t} = \left[\tilde{\mathbf{x}}_{i,t-L+1}^{\top}, \tilde{\mathbf{x}}_{i,t-L+2}^{\top}, \ldots, \tilde{\mathbf{x}}_{i,t}^{\top}\right]^{\top} \in \mathbb{R}^{Ld}, \tag{3}$$

and its target is the capped RUL value at the final cycle of the window. All admissible rolling windows are used for model development. For the official test split, a single sample is constructed per engine from the last available window. If a test trajectory is shorter than L, the first normalized observation is repeated on the left so that every engine yields one fixed-size input vector.

To make Eq. (3) concrete, consider a window length of $L = 5$ and a retained feature set of size $d = 4$. The input to the ANN at cycle t is then not a single 4-dimensional observation, but a 5×4 temporal block containing the normalized measurements from cycles $t-4$ through t. This block is flattened row by row into a single vector of length $Ld = 20$. The ANN therefore receives a short explicit time-delay embedding: the most recent cycle is placed at the end of the vector, while the earlier rows preserve the immediate degradation trend seen over the previous four cycles.

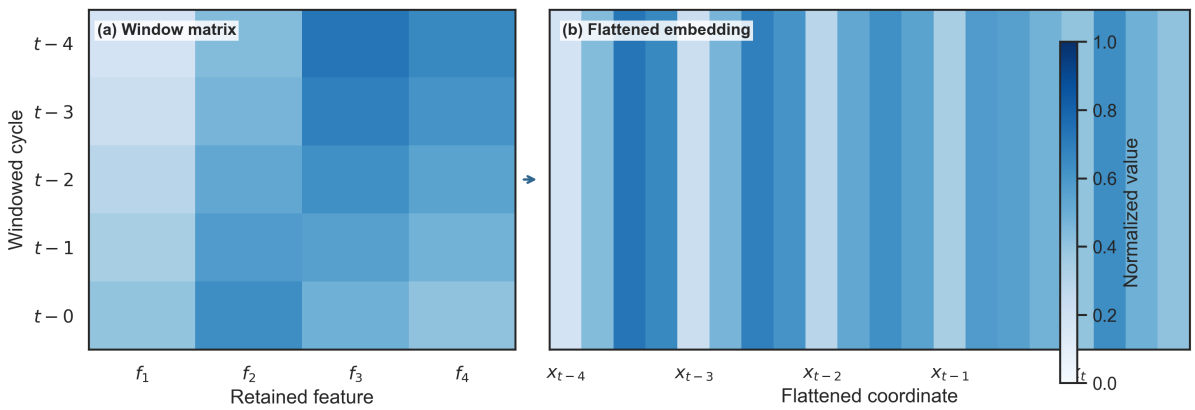

Figure 2: Illustrative construction of the flattened sliding-window embedding. A short matrix of normalized measurements is stacked over the last L cycles and then reshaped into a single ANN input vector.

2.1.5 PSO Screening, MILP-Guided Pruning, and Final Evaluation

The reported experiments use the measurement-only pruning branch built around a configurable feed-forward regressor. In the current configuration, the search space allows one or two hidden layers with 10–100 neurons per layer, ReLU activations, and an Adam optimizer. To keep the exact pruning stage computationally tractable while still using enough variation for meaningful model selection, the present study uses a development fraction of 0.3 from the windowed training and validation samples. The shared experimental settings used to produce the figures and tables in this manuscript are summarized in Table 2.

Table 2: Common experimental configuration used by the measurement-only ANN + PSO + pruning study.

Setting	**Value**
Normalization	global_standard
Sequence length	30
RUL clip	125
Validation split	0.20
Training fraction	0.30
PSO particles	5
PSO iterations	10
Stage 1 epochs	10
Stage 1 patience	3
Top-k candidates	3
Stage 3 tuning epochs	120
Stage 3 tuning patience	12
Final retraining epochs	100
Pruning keep fraction	0.10
Calibration subset size	16
MILP time limit (s)	30.0
Two-layer pruning strategy	reduced neighborhood

The workflow is explicitly divided into three model-selection stages plus one final benchmark stage. In Stage 1, PSO is used only as a low-cost structural screening mechanism. The official NASA train/test partition is preserved, and model selection is performed only inside the official training split. More specifically, the engine units in `train_FD00x.txt` are divided into internal training and validation groups using a fixed random seed before rolling windows are constructed, so that all windows from a given engine remain on one side of the split. This choice avoids leakage between highly overlapping windows from the same trajectory while preserving comparability with papers that report final results on the official benchmark test split. Each particle is decoded into a discrete ANN architecture by rounding and clipping the continuous position vector to the configured bounds. Candidate models are trained under a common reduced budget, and their low-fidelity objective is defined as

$$J_{\mathrm{LF}} = \mathrm{MSE}_{\mathrm{val}} + \lambda_{\mathrm{comp}}\, P_{\mathrm{comp}}, \tag{4}$$

with

$$P_{\mathrm{comp}} = N_{\mathrm{layers}} + \frac{N_{\mathrm{params}}}{1000}. \tag{5}$$

Here, $\mathrm{MSE}_{\mathrm{val}}$ is the validation mean squared error, N_{layers} is the number of hidden layers, and N_{params} is the number of trainable parameters. The complexity term discourages larger architectures unless they deliver meaningful validation gains. Under this design, PSO is not used to claim final global optimality; instead, it serves to reduce the structural search space at low computational cost.

In Stage 2, only the top-k Stage 1 structures are converted into cheap dense reference models and passed to the pruning stage. The pruning optimization does not directly minimize final RUL loss on the validation or test split. Instead, it operates on a small

calibration subset and minimizes the absolute deviation between the pruned ANN output y^n and the dense teacher output $\hat{y}^n$:

$$\min \sum_{n=1}^{N} e^n \tag{6}$$

subject to

$$e^n \geq y^n - \hat{y}^n, \qquad e^n \geq -(y^n - \hat{y}^n), \qquad e^n \geq 0. \tag{7}$$

For one-hidden-layer networks, the arc-selection problem is solved as an exact MILP with a standard big-M ReLU linearization. For two-hidden-layer networks, the default strategy first constructs an activation-aware heuristic mask, improves it with a bounded local search, fixes obviously active or inactive arcs, and then solves a reduced-neighborhood exact MILP over the remaining free binaries. If the local solver does not return a feasible incumbent within the time limit, the workflow falls back to a transparent magnitude-based pruning mask so that the end-to-end experiment still completes reproducibly.

In Stage 3, the pruned candidates are fine-tuned with a larger gradient-based budget and ranked on the same held-out validation engines. This stage is distinct from Stage 2: the pruning model reduces structure on a calibration subset, whereas Stage 3 re-optimizes predictive performance under the validation protocol used for model selection. Finally, the selected architecture is retrained in two forms on the full official training split: a dense reference ANN and its pruned counterpart. The official benchmark test split is then evaluated once for those final dense and pruned models. This sequencing makes the methodological claim explicit: PSO reduces the structural search space cheaply, pruning reduces structural redundancy on shortlisted candidates, and final model comparison is still based on validation-driven selection followed by one-time official-test evaluation. The complete protocol is summarized in Algorithm 1.

2.1.6 Pruning Optimization Model

The pruning stage is formulated as a teacher-matching mixed-integer optimization problem over a calibration subset. Let $\mathcal{N} = \{1, \ldots, N\}$ index calibration samples, $\mathcal{I} = \{1, \ldots, d\}$ the flattened input features, $\mathcal{H}_1 = \{1, \ldots, H_1\}$ the first hidden layer, $\mathcal{H}_2 = \{1, \ldots, H_2\}$ the optional second hidden layer, and $\mathcal{O} = \{1\}$ the scalar RUL output. For each sample $n \in \mathcal{N}$, let x_i^n denote input feature i and let $\hat{y}^n$ denote the output of the dense reference ANN on the same sample. The dense reference weights and biases are treated as fixed data. For one-hidden-layer networks these parameters are $\{W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}\}$; for two-hidden-layer networks they are $\{W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}, W^{(3)}, b^{(3)}\}$. Valid lower and upper bounds for each pre-activation and hidden activation are computed offline from the dense network and the calibration subset and are denoted generically by L and U.

Binary arc-selection variables indicate whether each dense connection is preserved in the sparse network. For the one-hidden-layer case,

$$z_{ji}^{(1)} \in \{0, 1\}, \qquad j \in \mathcal{H}_1,\ i \in \mathcal{I}, \tag{8}$$

$$z_{j}^{(2)} \in \{0, 1\}, \qquad j \in \mathcal{H}_1, \tag{9}$$

where $z_{ji}^{(1)} = 1$ keeps input-hidden arc (i, j) and $z_j^{(2)} = 1$ keeps hidden-output arc $(j, 1)$. For two-hidden-layer networks, an additional family

$$z_{kj}^{(2)} \in \{0, 1\}, \qquad k \in \mathcal{H}_2,\ j \in \mathcal{H}_1, \tag{10}$$

is introduced for first-hidden to second-hidden arcs, while the output-arc variables become

$$z_k^{(3)} \in \{0,1\}, \qquad k \in \mathcal{H}_2. \tag{11}$$

Continuous variables represent pre-activations a, post-ReLU activations h, auxiliary products q, predicted outputs y^n, and absolute teacher-matching errors $e^n \geq 0$. Binary variables δ encode the active branch of each ReLU unit.

The common objective minimizes the deviation between the pruned network and the dense teacher:

$$\min \sum_{n \in \mathcal{N}} e^n. \tag{12}$$

For each sample n, the absolute-deviation linearization is

$$e^n \geq y^n - \hat{y}^n, \qquad e^n \geq -\left(y^n - \hat{y}^n\right), \qquad e^n \geq 0. \tag{13}$$

For a one-hidden-layer ReLU ANN, the exact MILP is:

$$a_j^{n,1} = \sum_{i \in \mathcal{I}} W_{ji}^{(1)} x_i^n z_{ji}^{(1)} + b_j^{(1)}, \qquad n \in \mathcal{N},\ j \in \mathcal{H}_1, \tag{14}$$

$$h_j^{n,1} \geq a_j^{n,1}, \qquad h_j^{n,1} \geq 0, \tag{15}$$

$$h_j^{n,1} \leq a_j^{n,1} - L_j^{n,1}\left(1 - \delta_j^{n,1}\right), \qquad h_j^{n,1} \leq U_j^{n,1} \delta_j^{n,1}, \tag{16}$$

$$0 \leq q_j^n \leq U_j^{n,1} z_j^{(2)}, \qquad q_j^n \leq h_j^{n,1}, \qquad q_j^n \geq h_j^{n,1} - U_j^{n,1}\left(1 - z_j^{(2)}\right), \tag{17}$$

$$y^n = \sum_{j \in \mathcal{H}_1} W_j^{(2)} q_j^n + b^{(2)}, \qquad n \in \mathcal{N}. \tag{18}$$

The auxiliary variable q_j^n linearizes the product between the hidden activation and the binary decision that keeps or removes the outgoing arc.

The two-hidden-layer exact model extends these equations by introducing a second ReLU block:

$$a_k^{n,2} = \sum_{j \in \mathcal{H}_1} W_{kj}^{(2)} q_{kj}^{n,12} + b_k^{(2)}, \qquad n \in \mathcal{N},\ k \in \mathcal{H}_2, \tag{19}$$

$$0 \leq q_{kj}^{n,12} \leq U_j^{n,1} z_{kj}^{(2)}, \qquad q_{kj}^{n,12} \leq h_j^{n,1}, \qquad q_{kj}^{n,12} \geq h_j^{n,1} - U_j^{n,1}\left(1 - z_{kj}^{(2)}\right), \tag{20}$$

$$h_k^{n,2} \geq a_k^{n,2}, \qquad h_k^{n,2} \geq 0, \tag{21}$$

$$h_k^{n,2} \leq a_k^{n,2} - L_k^{n,2}\left(1 - \delta_k^{n,2}\right), \qquad h_k^{n,2} \leq U_k^{n,2} \delta_k^{n,2}, \tag{22}$$

$$0 \leq q_k^{n,3} \leq U_k^{n,2} z_k^{(3)}, \qquad q_k^{n,3} \leq h_k^{n,2}, \qquad q_k^{n,3} \geq h_k^{n,2} - U_k^{n,2}\left(1 - z_k^{(3)}\right), \tag{23}$$

$$y^n = \sum_{k \in \mathcal{H}_2} W_k^{(3)} q_k^{n,3} + b^{(3)}, \qquad n \in \mathcal{N}. \tag{24}$$

Let $\mathcal{A}$ denote the full set of dense arcs. With pruning ratio $\alpha \in (0,1)$, the kept-arc budget is controlled by

$$\sum_{a \in \mathcal{A}} z_a \leq (1 - \alpha)\,|\mathcal{A}|, \tag{25}$$

or by equality when an exact keep budget is required. In the current experiments, the pruning branch fixes a target keep ratio and applies Eq. (25) accordingly.

For two-hidden-layer networks, the full exact MILP is often too large for the available time budget. The implemented reduced-neighborhood strategy therefore first constructs an activation-aware heuristic mask and partitions the arc set into three groups: arcs fixed to keep, $\mathcal{A}_{\text{keep}}$; arcs fixed to drop, $\mathcal{A}_{\text{drop}}$; and uncertain arcs, $\mathcal{A}_{\text{free}}$. The exact model is then solved only over $\mathcal{A}_{\text{free}}$, with

$$z_a = 1 \quad \forall a \in \mathcal{A}_{\text{keep}}, \qquad z_a = 0 \quad \forall a \in \mathcal{A}_{\text{drop}}, \tag{26}$$

while the remaining binaries still satisfy the same activation and budget constraints. This matheuristic strategy preserves the structure of the exact model but concentrates the branch-and-bound effort on a smaller neighborhood around a strong activation-aware seed.

The heuristic seed is not purely magnitude-based. For each dense layer ℓ, let $u_i^{n,\ell-1}$ denote the incoming activation to coordinate i for calibration sample n, and define the mean absolute incoming activation

$$\bar{u}_i^{\ell-1} = \frac{1}{N} \sum_{n \in \mathcal{N}} \left| u_i^{n,\ell-1} \right|. \tag{27}$$

The activation-aware score of arc (i, j) in layer ℓ is then

$$s_{ji}^{(\ell)} = \left| W_{ji}^{(\ell)} \right| \bar{u}_i^{\ell-1}. \tag{28}$$

These scores are flattened across all prunable layers and sorted globally. If K arcs must be kept under the target pruning ratio, the initial heuristic mask $m^{(0)}$ keeps the K arcs with highest s_a values and drops the remainder. The local-search stage then improves this seed by same-layer swap moves that preserve the keep budget. Specifically, each round builds a candidate set from the weakest currently kept arcs and the strongest currently dropped arcs, and evaluates swaps according to the teacher-matching loss. If a_{keep} is an active arc and a_{drop} is an inactive arc from the same layer, the score gain used to prioritize the move is

$$\Delta s = s_{a_{\text{drop}}} - s_{a_{\text{keep}}}. \tag{29}$$

Only a bounded number of the best-ranked swaps are evaluated, and a candidate mask is accepted when it decreases the teacher-matching mean absolute error on the calibration subset.

After local search, the uncertain neighborhood is defined relative to the global score threshold τ of the final keep mask. For each arc a,

$$u_a = |s_a - \tau|, \tag{30}$$

so arcs with scores close to the threshold are the most ambiguous. The implementation also forces into $\mathcal{A}_{\text{free}}$ any arcs whose state changed between the initial top-K mask and the local-search-improved mask. The exact MILP then optimizes only this uncertain band, while the other arc decisions remain fixed. The main hyperparameters of this reduced-neighborhood matheuristic are summarized in Table 3.

Table 3: Parameters of the activation-aware reduced-neighborhood heuristic used before the exact two-hidden-layer MILP.

Symbol	**Meaning**	**Value**
α	Global pruning ratio	0.90
ρ_{free}	Free-arc fraction before clipping	0.10
$F_{\min}$	Minimum free binary arcs	1000
$F_{\max}$	Maximum free binary arcs	4000
R_{LS}	Local-search rounds	4
P_{LS}	Swap pool size per side	6
$E_{\max}$	Maximum swap evaluations per round	32
Time limit	MILP time limit per solve (s)	30.0
Calibration	Teacher-matching subset size	16

Algorithm 1 PSO screening, MILP-guided pruning, and final evaluation for C-MAPSS RUL prediction

Require: Dataset subset $D \in \{\mathrm{FD001}, \mathrm{FD002}, \mathrm{FD003}, \mathrm{FD004}\}$, sequence length L, training RUL cap $R_{\max}$, feature policy, normalization mode, PSO settings, pruning settings, tuning settings

Ensure: Final dense and pruned ANNs plus one-time official test metrics on `test_`D

```
Load train_D and compute capped training RUL targets using Eq. (2)
Apply dataset-aware feature filtering and fit the search-stage scaler on internal training engines only
Split the engine units in train_D into internal training units and validation units
Construct rolling windows of length L for internal training and validation units using Eq. (3)
for t = 1 to T_PSO do
    for each particle p in the swarm do
        Decode p into a valid ANN architecture by rounding and clipping the particle position
        if the candidate is invalid then
            Assign infinite objective and continue
        end if
        Build the ANN and train it under the low-fidelity screening budget
        Evaluate validation MSE on the held-out validation engines
        Compute J_LF using Eq. (4)
        Update particle-best and global-best records
    end for
    Update swarm velocities and positions
end for
Select the top-k Stage 1 architectures according to J_LF
for each shortlisted architecture do
    Train a cheap dense reference ANN on the internal training split
    Sample a calibration subset from the internal training windows
    Solve the pruning model using Eq. (6); if the exact two-layer solve times out, use the fallback pruning mask
    Fine-tune the pruned ANN on the internal training split
    Evaluate the pruned ANN on the held-out validation engines
end for
Select the best pruned candidate using the validation score
Refit the feature scaler on the full official training split
Retrain a final dense reference ANN on the full official training split
Rebuild the selected pruning mask on the full official training split and fine-tune the final pruned ANN
Load test_D and RUL_D only after model selection is complete
Apply the full-training scaler to the official test split and build one final window per test engine
Evaluate the final dense and pruned ANNs once on the official benchmark test split
```

A la metodología le falta bastante trabajo, es acá donde se presentará la novedad del paper, sugiero darle prioridad

3 Results and Diagnostics

The results reported here come from the experimental ANN + PSO + pruning branch executed on the four standard C-MAPSS subsets with a shared configuration. Table 4 summarizes the dense-versus-pruned comparison for the final selected architecture in each subset, while Table 5 reports the associated topology and compression statistics.

Los acrónimos se deberían definir solo una vez, dado que es un paper experimentar probablemente se pida comparar cada etapa con algún baseline, por ejemplo, comparar la selección de hiperparámetros con grid search cross validation o con Bayesian Optimization. Ahí se podría evaluar tiempo computacional y métricas de rendimiento. Por otra parte, se debería comparar la performance del modelo final con alguna red más grande o con Random Forest. Por último la fase final de predicción también se debería comparar con el control con PSO.

Table 4: Dense-versus-pruned performance for the selected architecture in each C-MAPSS subset. Validation metrics come from the internal held-out engine split; official-test metrics use the benchmark test partition once at the end.

Dataset	Dense Val. MSE	Pruned Val. MSE	Dense Test MSE	Pruned Test MSE	Dense Test MAE	Pruned Test MAE
FD001	464.43	256.76	300.80	318.84	13.41	13.18
FD002	963.84	787.80	1302.35	1398.66	26.94	27.89
FD003	287.68	211.78	249.70	296.20	12.04	13.53
FD004	654.86	642.36	1413.67	1470.83	30.23	31.45

Table 5: Selected dense topologies and resulting pruning compression for each subset. The reported pruning method corresponds to the final structure actually used in the selected candidate.

Dataset	Layers	Widths	Dense Params.	Nonzero Params.	Active Arcs	Keep Ratio	Method
FD001	2	98-87	38199	3987	3801	0.100	Fallback magnitude
FD002	2	100-23	41447	4256	4132	0.100	Fallback magnitude
FD003	2	100-25	32651	3378	3252	0.100	Fallback magnitude
FD004	2	100-17	40835	4190	4072	0.100	Fallback magnitude

Two immediate patterns emerge from Table 4. First, the pruned models improve validation MSE in all four subsets relative to their dense references. Second, that validation gain does not translate into a better official-test score under the current pruning budget: the dense model remains better on the official test split for FD001–FD004. This validation–test divergence is particularly visible in FD002 and FD004, which contain six operating regimes and therefore place a heavier burden on generalization across engine conditions.

Figure 3 shows the Stage 1 PSO convergence traces across the four subsets. The swarm improves rapidly during the first iterations and then enters a flatter regime, which is consistent with the intended role of PSO in this study: not to certify a global optimum, but to identify a compact set of architectures worth carrying forward into the pruning and retuning stages.

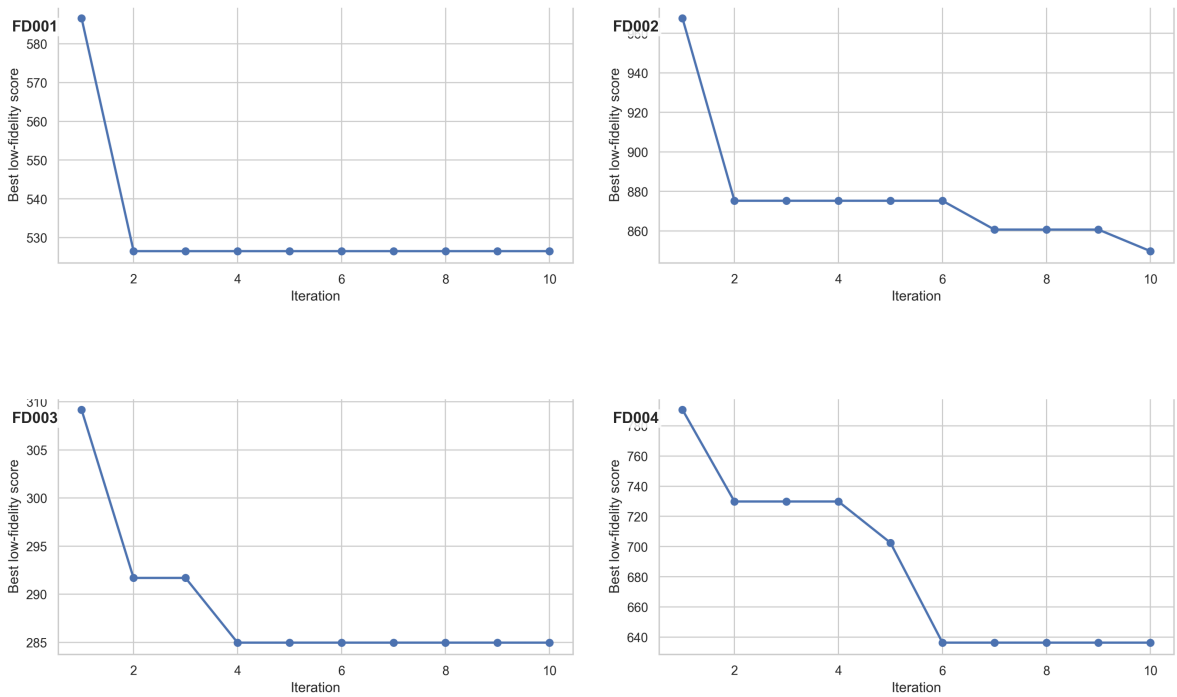

Figure 3: Stage 1 PSO convergence across FD001–FD004. Each panel summarizes the best low-fidelity score reached by the swarm for one subset.

The dense-versus-pruned trade-off is easier to interpret in aggregate in Figure 4. The bottom-left panel confirms that pruning reduces validation MSE for all four subsets, but the top row shows that the official-test MSE and MAE remain slightly lower for the dense references. The bottom-right panel makes the computational trade-off explicit: once pruning and fine-tuning are added, the pruned branch becomes more expensive than dense final retraining in the current configuration, even though the resulting network is much smaller.

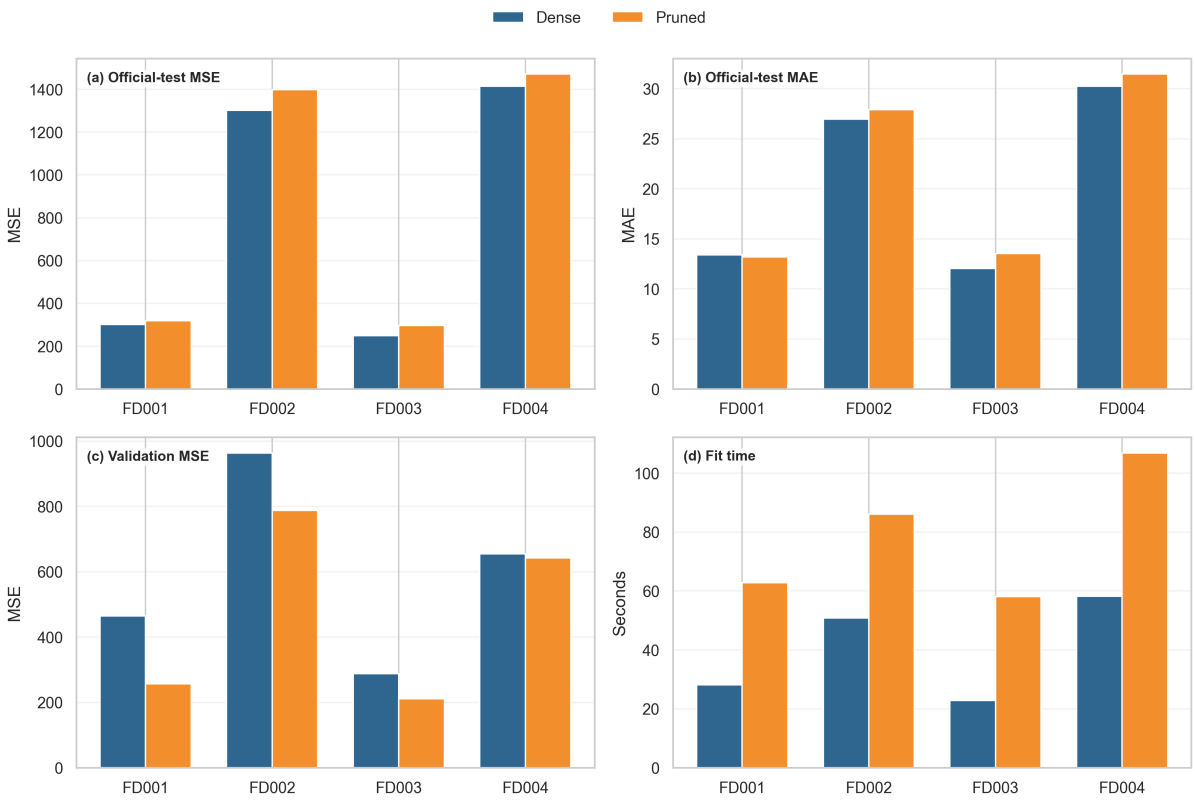

Figure 4: Aggregate dense-versus-pruned comparison across the four subsets. The figure contrasts validation error, official-test error, and fit time under the shared experimental configuration.

The training dynamics of the final selected pruned candidates are shown in Figure 5. The histories are smooth and monotone on the training split, while the validation curves flatten well before the maximum epoch budget in the easier subsets. This behavior suggests that the current late-stage budgets are sufficient for candidate selection and that simply extending the epoch count is unlikely to close the dense-versus-pruned official-test gap without changing the pruning budget or the final fine-tuning strategy.

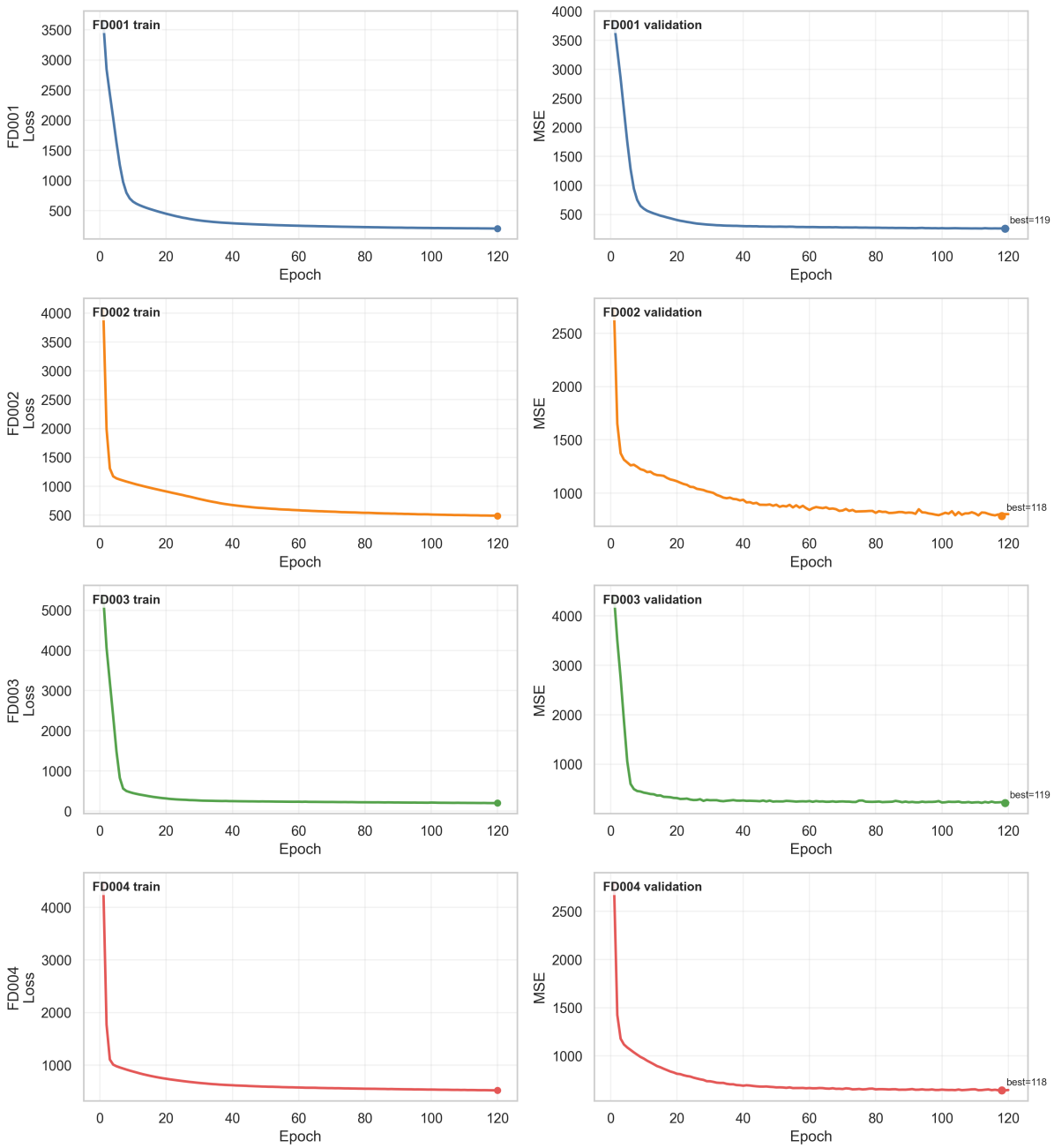

Figure 5: Stage 3 training histories for the final selected pruned candidate in each subset. Each row corresponds to one C-MAPSS subset, with training loss on the left and validation MSE on the right.

Finally, Figure 6 compares the official-test prediction behavior across all subsets using the complete benchmark test split for each dataset. Each point corresponds to one engine, so the panels summarize the full set of official test engines rather than a single illustrative trajectory. FD001 and FD003 remain the most stable cases, whereas FD002 and FD004 exhibit visibly wider spread in the predicted RUL values. This is consistent with the subset-level descriptive statistics: the multi-regime datasets combine operating-condition variability with truncated test trajectories and remain the most difficult setting for lightweight ANN baselines.

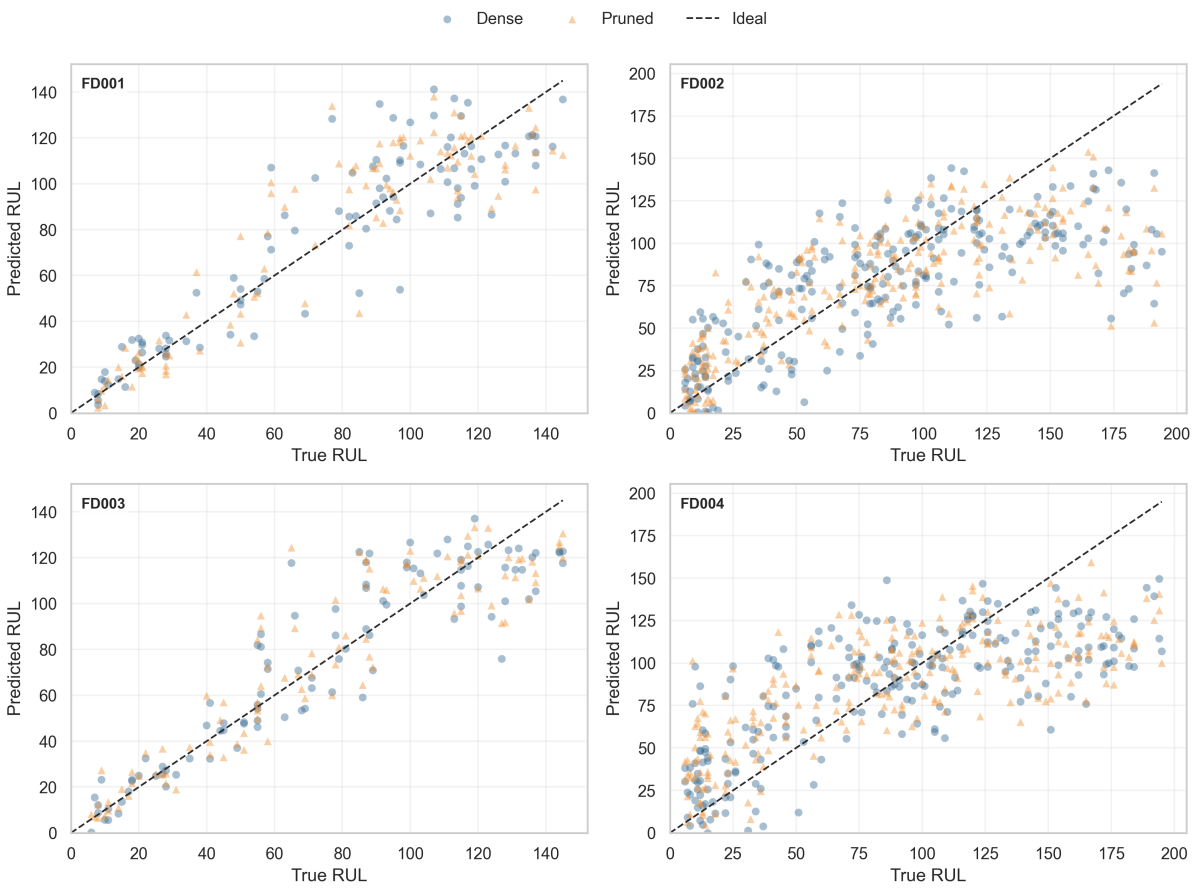

Figure 6: Official-test prediction scatterplots for dense and pruned models across FD001–FD004. Each semi-transparent point represents one engine from the complete official NASA test split, and the diagonal indicates ideal agreement between predicted and true RUL.

4 Discussion

The experiments support two conclusions that are easy to miss if only one metric is considered. On the one hand, the pruning stage is not meaningless: under the current 10% keep-ratio budget, the selected pruned models consistently improve internal validation MSE, which indicates that the pruning-plus-fine-tuning stage acts as a strong structural regularizer on the held-out development engines. On the other hand, the official benchmark test set reveals that this regularization is too aggressive at the present compression level. In all four subsets, the dense reference remains the better final predictor once the official NASA split is consulted.

The topology table adds another important diagnostic. All four selected final architectures are two-hidden-layer networks, and the reported pruning method for each subset is a magnitude-based fallback after attempting the reduced-neighborhood exact MILP. This outcome is methodologically informative rather than merely inconvenient. It shows that the reduced-neighborhood exact formulation meaningfully shrinks the binary search space, but still struggles to deliver a feasible incumbent fast enough for the present 30-second limit and the current layer widths. In other words, the mathematical

pruning model is useful as a guide for structured compression, yet exact sparse-structure optimization remains a computational bottleneck at this scale.

The subset pattern is also consistent with the descriptive statistics in the data card. FD001 and FD003 are single-regime datasets with more homogeneous operating settings, and they remain easier for lightweight MLPs. FD002 and FD004 require generalization across multiple operating regimes, and their higher official-test errors confirm that regime diversity is a stronger challenge for this family of models than simply adding a second fault mode. This makes the current workflow especially valuable as a measurement study: it separates the effects of data complexity, model size, and optimization strategy without relying on test-set reuse.

5 Conclusion

This paper reframed the project as a measurement-only RUL study and evaluated a reproducible ANN workflow built around low-cost PSO screening, MILP-guided pruning, and selective retuning on the NASA C-MAPSS benchmark. The main methodological contribution is not a claim of global optimality, but a disciplined separation of roles: PSO screens structures cheaply, pruning compresses shortlisted models on a calibration subset, and final evaluation remains tied to a single use of the official NASA test split.

Under the current aggressive pruning budget, the pruned candidates retain roughly one tenth of the active arcs and improve validation error, but the dense reference still provides the best official-test accuracy across FD001–FD004. The results therefore motivate two immediate next steps: less aggressive keep ratios and stronger exact-pruning neighborhoods for two-hidden-layer models. Even with that limitation, the workflow provides a scientifically defensible baseline for studying how lightweight architecture search and structured pruning interact in turbofan RUL prediction.

References

[1] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *2008 International Conference on Prognostics and Health Management*, pp. 1–9, IEEE, 2008.

[2] Y. Lei, N. Li, L. Guo, N. Li, T. Yan, and J. Lin, "Machinery health prognostics: A systematic review from data acquisition to rul prediction," *Mechanical Systems and Signal Processing*, vol. 104, pp. 799–834, 2018.

[3] Y. Wu, M. Yuan, S. Dong, L. Lin, and Y. Liu, "Remaining useful life estimation of engineered systems using vanilla lstm neural networks," *Neurocomputing*, vol. 275, pp. 167–179, 2018.

[4] A. Peringal *et al.*, "Temporal degradation modeling in turbofan engines using recurrent networks," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 2, pp. 1122–1135, 2024.

[5] M. Elsherif *et al.*, "Spatial-temporal feature extraction for remaining useful life prediction via caelstm," *Reliability Engineering & System Safety*, vol. 230, p. 108923, 2025.

[6] Z. Chen *et al.*, "Position-sensitive attention in transformer architectures for prognostics," *Applied Soft Computing*, vol. 150, p. 110123, 2024.

[7] Y. Mo, Q. Wu, X. Li, and B. Huang, "Remaining useful life estimation via transformer encoder with self-attention," *IEEE Transactions on Instrumentation and Measurement*, vol. 70, pp. 1–10, 2021.

[8] L. Yang and A. Shami, "On hyperparameter optimization of machine learning algorithms: Theory and practice," *Neurocomputing*, vol. 415, pp. 295–316, 2020.

[9] J. Snoek, H. Larochelle, and R. P. Adams, "Practical bayesian optimization of machine learning algorithms," in *Advances in Neural Information Processing Systems*, vol. 25, pp. 2951–2959, 2012.

[10] Y. Xue *et al.*, "Automated architecture search using bayesian optimization for turbofan engine rul estimation," *Knowledge-Based Systems*, vol. 290, p. 111534, 2025.

[11] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green ai," *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.

[12] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in nlp," *arXiv preprint arXiv:1906.02243*, 2019.

[13] E. Garcia-Martin, C. F. Rodrigues, G. Riley, and H. Grahn, "Estimation of energy consumption in machine learning," *Journal of Parallel and Distributed Computing*, vol. 134, pp. 75–88, 2019.

[14] P. Dhar, S. Bhattacharya, *et al.*, "A survey on the carbon footprint of deep learning formulations," *IEEE Access*, 2021.

[15] C.-j. Wu *et al.*, "Sustainable ai: Environmental implications, challenges and opportunities," *Patterns*, vol. 3, no. 1, 2022.

[16] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "Carbon emissions and large neural network training," *arXiv preprint arXiv:2104.10350*, 2021.

[17] J. Li *et al.*, "Edge intelligence: Architectures, challenges, and applications," *IEEE Network*, 2023.

[18] M. Merenda, C. Porcaro, and D. Iero, "Edge machine learning for sensor networks in iot era: A review," *Sensors*, vol. 20, no. 8, p. 2275, 2020.

[19] X. Wang, Y. Han, V. C. Leung, D. Niyato, X. Yan, and X. Chen, "Convergence of edge computing and deep learning: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 869–904, 2020.

[20] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.

[21] C. Jiang *et al.*, "Model compression and edge intelligence," *IEEE Internet of Things Journal*, 2021.

[22] J. Chen and X. Ran, "Deep learning with edge computing: A review," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1655–1674, 2019.

[23] J. Gong *et al.*, "Robust fractional-order pid controller design under shifting operational regimes," *Control Engineering Practice*, vol. 130, p. 105372, 2023.

A Ideas

A.1 MILP to prone NNs

Sets and indices

$$\mathcal{N} = \{1, \ldots, N\} \quad \text{training samples,} \tag{31}$$
$$\mathcal{H} = \{1, \ldots, H\} \quad \text{hidden neurons,} \tag{32}$$
$$\mathcal{I} = \{1, \ldots, d\} \quad \text{input features,} \tag{33}$$
$$\mathcal{O} = \{1, \ldots, m\} \quad \text{output neurons.} \tag{34}$$

Given data

$$x_i^n : \text{ input feature } i \text{ of sample } n, \tag{35}$$
$$\hat{y}_k^n : \text{ output of the original (reference) neural network for sample } n \text{ and output } k. \tag{36}$$

Let $W_{ji}^{(1)}$, $b_j^{(1)}$ be the weights and biases from input to hidden layer, and $W_{kj}^{(2)}$, $b_k^{(2)}$ the weights and biases from hidden to output layer of the pruned network before arc deletion. We also define big-M constants for the ReLU formulation:

$$L_j^n \leq a_j^n \leq U_j^n, \qquad \forall n \in \mathcal{N},\ j \in \mathcal{H},$$

where a_j^n is the pre-activation of hidden neuron j for sample n, and L_j^n, U_j^n are valid lower and upper bounds.

Decision variables

$$z_{ji}^{(1)} \in \{0, 1\} \qquad \forall j \in \mathcal{H},\ i \in \mathcal{I},$$

$$z_{kj}^{(2)} \in \{0, 1\} \qquad \forall k \in \mathcal{O},\ j \in \mathcal{H},$$

where $z_{ji}^{(1)} = 1$ if arc (i, j) is kept, and $z_{kj}^{(2)} = 1$ if arc (j, k) is kept.

For each sample n:

$$a_j^n \in \mathbb{R} \qquad \forall j \in \mathcal{H},$$
$$h_j^n \geq 0 \qquad \forall j \in \mathcal{H},$$
$$\delta_j^n \in \{0, 1\} \qquad \forall j \in \mathcal{H},$$
$$y_k^n \in \mathbb{R} \qquad \forall k \in \mathcal{O}.$$

Objective

Minimize the quadratic error between the pruned network and the reference network:

$$\min \sum_{n \in \mathcal{N}} \sum_{k \in \mathcal{O}} \left(y_k^n - \hat{y}_k^n\right)^2 .$$

Layer equations

Hidden-layer pre-activation:

$$a_j^n = \sum_{i \in \mathcal{I}} W_{ji}^{(1)} \, x_i^n \, z_{ji}^{(1)} + b_j^{(1)}, \qquad \forall n \in \mathcal{N},\ j \in \mathcal{H}.$$

ReLU activation $h_j^n = \max\{0, a_j^n\}$, linearized with big-M:

$$h_j^n \geq a_j^n, \qquad \forall n \in \mathcal{N},\ j \in \mathcal{H},$$

$$h_j^n \geq 0, \qquad \forall n \in \mathcal{N},\ j \in \mathcal{H},$$

$$h_j^n \leq a_j^n - L_j^n(1 - \delta_j^n), \qquad \forall n \in \mathcal{N},\ j \in \mathcal{H},$$

$$h_j^n \leq U_j^n \delta_j^n, \qquad \forall n \in \mathcal{N},\ j \in \mathcal{H}.$$

Output layer:

$$y_k^n = \sum_{j \in \mathcal{H}} W_{kj}^{(2)} \, h_j^n \, z_{kj}^{(2)} + b_k^{(2)}, \qquad \forall n \in \mathcal{N},\ k \in \mathcal{O}.$$

Pruning budget

Let the total number of arcs in the original two-layer network be

$$A_{\text{tot}} = |\mathcal{H}||\mathcal{I}| + |\mathcal{O}||\mathcal{H}|.$$

If at least an α fraction of arcs must be deleted, then the number of kept arcs can be at most $(1-\alpha)A_{\text{tot}}$:

$$\sum_{j \in \mathcal{H}} \sum_{i \in \mathcal{I}} z_{ji}^{(1)} + \sum_{k \in \mathcal{O}} \sum_{j \in \mathcal{H}} z_{kj}^{(2)} \ \leq \ (1-\alpha)\, A_{\text{tot}}.$$

Equivalently, if exactly an α fraction of arcs must be deleted, impose:

$$\sum_{j \in \mathcal{H}} \sum_{i \in \mathcal{I}} z_{ji}^{(1)} + \sum_{k \in \mathcal{O}} \sum_{j \in \mathcal{H}} z_{kj}^{(2)} \ = \ (1-\alpha)\, A_{\text{tot}}.$$

Domain constraints

$$z_{ji}^{(1)} \in \{0,1\}, \qquad z_{kj}^{(2)} \in \{0,1\}, \qquad \delta_j^n \in \{0,1\},$$

$$a_j^n \in \mathbb{R}, \qquad h_j^n \geq 0, \qquad y_k^n \in \mathbb{R}.$$

Complete model

$$
\begin{aligned}
\min \quad & \sum_{n\in\mathcal{N}} \sum_{k\in\mathcal{O}} (y_k^n - \hat{y}_k^n)^2 \\
\text{s.t.} \quad & a_j^n = \sum_{i\in\mathcal{I}} W_{ji}^{(1)}\, x_i^n\, z_{ji}^{(1)} + b_j^{(1)}, && \forall n \in \mathcal{N},\ j \in \mathcal{H}, \\
& h_j^n \geq a_j^n, && \forall n \in \mathcal{N},\ j \in \mathcal{H}, \\
& h_j^n \geq 0, && \forall n \in \mathcal{N},\ j \in \mathcal{H}, \\
& h_j^n \leq a_j^n - L_j^n(1-\delta_j^n), && \forall n \in \mathcal{N},\ j \in \mathcal{H}, \\
& h_j^n \leq U_j^n \delta_j^n, && \forall n \in \mathcal{N},\ j \in \mathcal{H}, \\
& y_k^n = \sum_{j\in\mathcal{H}} W_{kj}^{(2)}\, h_j^n\, z_{kj}^{(2)} + b_k^{(2)}, && \forall n \in \mathcal{N},\ k \in \mathcal{O}, \\
& \sum_{j\in\mathcal{H}} \sum_{i\in\mathcal{I}} z_{ji}^{(1)} + \sum_{k\in\mathcal{O}} \sum_{j\in\mathcal{H}} z_{kj}^{(2)} \leq (1-\alpha)\, A_{\text{tot}}, \\
& z_{ji}^{(1)} \in \{0,1\}, && \forall j \in \mathcal{H},\ i \in \mathcal{I}, \\
& z_{kj}^{(2)} \in \{0,1\}, && \forall k \in \mathcal{O},\ j \in \mathcal{H}, \\
& \delta_j^n \in \{0,1\}, && \forall n \in \mathcal{N},\ j \in \mathcal{H}, \\
& a_j^n \in \mathbb{R},\ h_j^n \geq 0,\ y_k^n \in \mathbb{R}, && \forall n \in \mathcal{N},\ j \in \mathcal{H},\ k \in \mathcal{O}.
\end{aligned}
$$